El Modelo de Objeto del Documento (DOM) en JavaScript

1. Introducción al DOM

El **Modelo de Objeto del Documento** (Document Object Model), conocido como **DOM**, es un estándar que describe cómo se representa una página web dentro del navegador. Esta representación se organiza como un **árbol de nodos** en el que cada elemento, atributo y fragmento de texto del documento HTML se modela como un objeto con propiedades y métodos.

El DOM permite que el contenido generado por un lenguaje de marcas (como HTML) pueda:

- **Leerse**
- **Modificar su estructura**
- **Cambiar su estilo**
- **Responder a las acciones del usuario**

JavaScript utiliza el DOM como puente para interactuar dinámicamente con la página, logrando interfaces interactivas, animaciones, validaciones de formularios y experiencias ricas en tiempo real.

---

2. Componentes fundamentales del DOM

La API del DOM está organizada en torno a varios tipos de objetos. Cada uno desempeña una función específica dentro del árbol del documento.

2.1. El objeto `document`

`document` es el punto de acceso principal al DOM. Representa **todo el documento HTML** cargado en el navegador y permite:

- Buscar elementos,
- Crear y eliminar nodos,
- Consultar información del documento,
- Registrar eventos globales.

Es, por tanto, el “árbol completo” desde el cual se inicia cualquier operación de manipulación.

2.2. Objetos `Element`

Cada etiqueta del documento (como `<div>`, `<p>`, `<img>`, etc.) se representa mediante un objeto de tipo **Element**. Los elementos poseen:

- Contenido,
- Atributos,
- Estilos,
- Relaciones jerárquicas con otros nodos.

Permiten realizar modificaciones directas sobre los componentes visibles de la página.

2.3. Colecciones: `NodeList` y `HTMLCollection`

Ciertos métodos del DOM no devuelven un solo elemento, sino **colecciones**:

- **NodeList:** lista de nodos estática o dinámica según el método usado.
- **HTMLCollection:** colección “viva” que se actualiza automáticamente si cambia el contenido del DOM.

Estas colecciones permiten recorrer múltiples elementos y aplicar operaciones en grupo.

2.4. Objetos `Event`

Cada acción del usuario o del navegador (como un clic, la pulsación de una tecla, la carga del documento, etc.) se modela mediante objetos de tipo **Event**.
Estos objetos contienen información detallada sobre lo ocurrido y son fundamentales para crear interfaces interactivas.

---

3. Selección de elementos en el DOM

Antes de modificar cualquier parte del documento, es necesario **localizar los elementos** con los que se desea trabajar. La API del DOM ofrece diferentes métodos de selección adecuados a distintas necesidades.

3.1. Selección por ID: `getElementById()`

Permite obtener un elemento único identificado mediante su atributo `id`.
Es altamente eficiente y el método preferido cuando se conoce este identificador.

3.2. Selección por clase: `getElementsByClassName()`

Devuelve una colección con todos los elementos que comparten una misma clase CSS.
La colección se actualiza automáticamente si se añaden o eliminan elementos del DOM.

3.3. Selección por etiqueta: `getElementsByTagName()`

Recupera todos los elementos que utilizan una etiqueta determinada (por ejemplo, todos los `<p>` o `<li>`).

3.4. Selección mediante selectores CSS: `querySelector()`

Método muy flexible, ya que permite utilizar cualquier selector CSS.
Devuelve únicamente el **primer** elemento que coincida.

3.5. Selección múltiple: `querySelectorAll()`

Funciona de forma similar a `querySelector`, pero devuelve **todos** los elementos coincidentes en una NodeList.

---

4. Manipulación de elementos

Una de las funciones más comunes del DOM es la modificación del contenido y los atributos de los elementos.

4.1. Modificación del contenido

El DOM permite alterar el interior de un elemento utilizando dos propiedades principales:

- **`innerHTML`**: reemplaza el contenido completo interpretando código HTML.
- **`textContent`**: reemplaza únicamente el texto, sin interpretar etiquetas.

La elección depende de si se necesita generar contenido estructurado o solo texto.

4.2. Gestión de atributos

Los atributos HTML pueden modificarse mediante:

- **`setAttribute()`**, para añadir o modificar un atributo,
- **`removeAttribute()`**, para eliminarlo.

Existen además propiedades específicas para ciertos atributos comunes, como:

- `src` para imágenes,
- `href` para enlaces,
- `value` para campos de formulario.

4.3. Estilos y clases

Para modificar el estilo visual de un elemento se utilizan:

- La propiedad **`style`**, que actúa sobre estilos en línea.
- El objeto **`classList`**, que permite añadir, eliminar o alternar clases CSS de forma eficiente.

---

5. Creación y eliminación de nodos en el árbol del documento

5.1. Creación de elementos

El método `document.createElement()` permite crear nuevos elementos HTML que inicialmente no están visibles en la página, ya que solo existen en memoria.

5.2. Inserción de nodos

Para añadir un elemento al DOM es necesario insertarlo en la posición deseada mediante métodos como:

- **`appendChild()`**, que añade el elemento al final del nodo padre,
- **`insertBefore()`**, que permite especificar la posición exacta de inserción.

5.3. Clonación de nodos

El método `cloneNode()` crea una copia de un elemento:

- Superficial (solo el nodo),
- O profunda (incluyendo todos los nodos hijos).

5.4. Eliminación de nodos

Los nodos pueden eliminarse mediante:

- **`removeChild()`**, invocado desde el nodo padre,
- **`remove()`**, aplicado directamente sobre el elemento objetivo.

Estas operaciones permiten construir o desmontar estructuras completas de la interfaz en tiempo real.

---

6. Manejo de eventos

La interactividad es una parte esencial de las páginas web modernas, y los eventos son el mecanismo base para capturar y gestionar acciones del usuario o del sistema.

6.1. Registro de eventos

El método **`addEventListener()`** permite asociar una función a un evento específico.
Al producirse el evento, la función se ejecuta automáticamente.

6.2. Eliminación de eventos

Si en algún momento es necesario retirar un manejador previamente registrado, puede utilizarse **`removeEventListener()`**, siempre indicando la misma referencia de función original.

6.3. Emisión de eventos

Con **`dispatchEvent()`**, es posible generar eventos personalizados y lanzarlos manualmente.
Esto permite coordinar interacciones entre distintos componentes de la aplicación.

6.4. Propagación de eventos

Los eventos en el DOM se propagan a través del árbol siguiendo etapas:

- **Captura**, desde la raíz hasta el elemento objetivo,
- **Fase objetivo**, donde se ejecuta el evento directamente,
- **Burbujeo**, desde el elemento objetivo de vuelta a la raíz.

Comprender este ciclo es esencial para controlar con precisión el comportamiento de la interfaz.

---

7. Navegación por el árbol DOM

La API del DOM no solo permite seleccionar elementos concretos, sino también desplazarse a través de la estructura del documento.

7.1. Navegación ascendente

Propiedades como **`parentNode`** y el método **`closest()`** permiten acceder a los elementos antecesores en el árbol.

7.2. Navegación descendente

Los elementos hijos pueden consultarse mediante:

- **`childNodes`**, que incluye nodos de texto,
- **`children`**, que solo incluye elementos,
- **`firstChild`** y **`lastChild`**, para acceder a los primeros o últimos nodos hijos.

7.3. Navegación lateral

Los nodos hermanos se recorren mediante propiedades como:

- **`nextSibling`** y **`previousSibling`**, que incluyen nodos de texto,
- **`nextElementSibling`** y **`previousElementSibling`**, que trabajan únicamente con elementos HTML.

Estas herramientas son especialmente útiles cuando se trabaja con estructuras repetitivas o cuando se manipula contenido generado dinámicamente.

Métodos y propiedades que aparecen en el documento.

Función / Propiedad	Tipo	Parámetros	Descripción breve	Uso típico
addEventListener(evento, función)	Método	`evento`: string, `función`: callback	Registra un manejador de eventos.	Responder a interacción del usuario.
appendChild(nodo)	Método	`nodo`: Node	Inserta un nodo como último hijo.	Añadir elementos al DOM.
childNodes	Propiedad	–	Lista de nodos hijos (incluye texto).	Navegación descendente.
children	Propiedad	–	Lista de elementos hijos (solo elementos).	Navegación por elementos.
classList.add(nombre)	Método	`nombre`: string	Añade una clase CSS.	Gestionar clases CSS.
classList.remove(nombre)	Método	`nombre`: string	Elimina una clase.	Gestionar estilos dinámicamente.
classList.toggle(nombre)	Método	`nombre`: string	Alterna una clase.	Activar/desactivar estados.
cloneNode(profundo)	Método	`profundo`: boolean	Clona un nodo (con o sin hijos).	Duplicar elementos.
closest(selector)	Método	`selector`: string	Obtiene el ancestro más cercano que coincida.	Buscar padres específicos.
dispatchEvent(evento)	Método	`evento`: Event	Dispara un evento manualmente.	Eventos personalizados.

Función / Propiedad	Tipo	Parámetros	Descripción breve	Uso típico
document	Objeto	–	Representa el documento completo cargado.	Punto de entrada para acceder al DOM.
document.createElement (etiqueta)	Método	`etiqueta`: string	Crea un nuevo elemento HTML.	Generar contenido dinámico.
firstChild / lastChild	Propiedades	–	Devuelven el primer/último nodo hijo.	Acceso rápido a extremos.
getElementById(id)	Método	`id`: string	Devuelve el elemento con ese ID.	Selección rápida y única.
getElementsBy ClassName(clase)	Método	`clase`: string	Devuelve una *HTMLCollection* de elementos con esa clase.	Selección múltiple dinámica.
getElementsBy TagName(etiqueta)	Método	`etiqueta`: string	Devuelve una *HTMLCollection* de elementos con esa etiqueta.	Selección por tipo de elemento.
innerHTML	Propiedad	–	Obtiene o establece el HTML interno del elemento.	Modificar contenido con etiquetas.
insertBefore(nuevo, referencia)	Método	`nuevo`: Node, `referencia`: Node	Inserta el nodo antes del nodo referencia.	Control preciso de inserción.
parentNode	Propiedad	–	Devuelve el nodo padre.	Navegación ascendente.
querySelector(selector)	Método	`selector`: string (CSS)	Devuelve el primer elemento que coincide con el selector.	Búsquedas flexibles.

Función / Propiedad	Tipo	Parámetros	Descripción breve	Uso típico
querySelectorAll(selector)	Método	`selector`: string (CSS)	Devuelve una *NodeList* con todos los elementos coincidentes.	Selección múltiple avanzada.
remove()	Método	–	Elimina el nodo actual.	Borrado directo.
removeAttribute(nombre)	Método	`nombre`: string	Elimina un atributo.	Quitar atributos dinámicamente.
removeChild(nodo)	Método	`nodo`: Node	Elimina un nodo hijo.	Borrar nodos desde el padre.
removeEventListener(evento, función)	Método	`evento`: string, `función`: callback	Quita un manejador de evento.	Gestionar eventos dinámicamente.
setAttribute(nombre, valor)	Método	`nombre`: string, `valor`: string	Añade o modifica un atributo.	Manipulación genérica de atributos.
style	Propiedad	–	Accede a estilos en línea.	Modificar estilos directamente.
textContent	Propiedad	–	Obtiene o establece solo el texto interno.	Modificar texto sin HTML.

AMPLIACIÓN

Inserción y eliminación modernas del DOM

Función / Propiedad	Tipo	Parámetros	Descripción breve	Uso típico
append()	Método	nodos o cadenas	Inserta nodos o texto al final.	Añadir contenido sin restricciones.
prepend()	Método	nodos o cadenas	Inserta al principio del elemento.	Crear encabezados o contenido inicial.
before()	Método	nodos o cadenas	Inserta antes del nodo actual.	Colocar contenido adyacente.
after()	Método	nodos o cadenas	Inserta después del nodo actual.	Añadir elementos contiguos.
replaceWith()	Método	nodo o cadena	Sustituye el nodo por otro.	Modificar estructuras completas.
insertAdjacentHTML(pos, html)	Método	`"beforebegin"`, `"afterbegin"`, `"beforeend"`, `"afterend"`, html	Inserta HTML optimizado sin reconstruir el DOM.	Insertar grandes bloques de HTML.
insertAdjacentElement()	Método	pos, elemento	Inserta un *elemento* en una posición relativa.	Controlar inserciones precisas.
insertAdjacentText()	Método	pos, texto	Inserta texto sin interpretar HTML.	Añadir contenido textual rápido.

Métodos avanzados del interfaz Node

Función / Propiedad	Tipo	Parámetros	Descripción breve	Uso típico
replaceChild(nuevo, antiguo)	Método	dos nodos	Reemplaza un nodo hijo existente.	Actualizar partes del DOM.
contains(nodo)	Método	nodo	Comprueba si el nodo está dentro del actual.	Validar jerarquía de elementos.
hasChildNodes()	Método	–	Indica si un nodo tiene hijos.	Comprobación previa antes de iterar.
isEqualNode(nodo)	Método	nodo	Compara nodos por estructura.	Comparar duplicados.
isSameNode(nodo)	Método	nodo	Comprueba si es literalmente el mismo objeto.	Identificar referencias.
normalize()	Método	–	Fusiona nodos de texto contiguos.	Limpiar el DOM.
nodeType	Propiedad	–	Devuelve el tipo de nodo (1, 3…)	Diferenciar elementos de texto.
nodeName	Propiedad	–	Nombre del nodo (DIV, P, #text).	Depuración y diagnósticos.

Navegación avanzada del DOM

Función / Propiedad	Tipo	Parámetros	Descripción breve	Uso típico
parentElement	Propiedad	–	Devuelve el padre, solo si es elemento.	Evitar nodos de texto.
firstElementChild	Propiedad	–	Primer hijo de tipo elemento.	Navegación por elementos.
lastElementChild	Propiedad	–	Último hijo de tipo elemento.	Acceso rápido.
childElementCount	Propiedad	–	Número de hijos elemento.	Contar elementos sin nodos textuales.
previousElementSibling	Propiedad	–	El hermano anterior tipo elemento.	Navegación horizontal.
nextElementSibling	Propiedad	–	El hermano siguiente tipo elemento.	Navegación horizontal limpia.
baseURI	Propiedad	–	URL base del documento.	Resolución de rutas internas.
ownerDocument	Propiedad	–	Documento al que pertenece el nodo.	Manejo avanzado de DOM.

Manejo avanzado de atributos

Función / Propiedad	Tipo	Parámetros	Descripción breve	Uso típico
getAttribute(nombre)	Método	nombre: string	Obtiene un atributo.	Leer atributos dinámicos.
hasAttribute(nombre)	Método	nombre: string	Verifica si existe el atributo.	Validación.
getAttributeNode(nombre)	Método	nombre: string	Obtiene el nodo Attr asociado.	Manipulación de atributos avanzada.
setAttributeNode(nodoAttr)	Método	nodo Attr	Inserta un nodo de atributo.	Control fino sobre Attr.
removeAttributeNode(attr)	Método	nodo Attr	Elimina un nodo de atributo.	Limpieza avanzada de atributos.
toggleAttribute(nombre)	Método	nombre	Activa o desactiva un atributo.	Utilizado en atributos booleanos (ej: disabled).
dataset	Propiedad	–	Acceso a los atributos data-*	Crear configuraciones personalizadas.
id / className	Propiedades	–	Permiten leer o modificar id y clase.	Cambios rápidos de identificación.

Eventos: herramientas avanzadas

Función / Propiedad	Tipo	Parámetros	Descripción breve	Uso típico
preventDefault()	Método	–	Cancela el comportamiento por defecto.	Formularios, enlaces, botones.
stopPropagation()	Método	–	Detiene la burbuja del evento.	Evitar comportamientos indeseados.
stopImmediatePropagation()	Método	–	Bloquea futuros listeners.	Control de prioridades.
event.target	Propiedad	–	Elemento donde se originó el evento.	Delegación.
event.currentTarget	Propiedad	–	Elemento que ejecuta el listener.	Distinción target/listener.
event.type	Propiedad	–	Tipo de evento (click, input…).	Depuración y lógicas de control.
event.timeStamp	Propiedad	–	Momento en que se generó el evento.	Tiempos de acciones.
CustomEvent()	Constructor	tipo, opciones	Crea eventos personalizados.	Comunicación entre componentes.

Propiedades importantes del objeto `document`

Función / Propiedad	Tipo	Parámetros	Descripción breve	Uso típico
document.body	Propiedad	–	Devuelve el elemento `<body>`.	Inserción de contenido global.
document.head	Propiedad	–	Devuelve el `<head>`.	Insertar scripts/meta dinámicos.
document.title	Propiedad	–	Lector/escritor del título.	Cambiar título dinámico.
document.readyState	Propiedad	–	Estado del documento (loading, complete).	Detectar carga.
document.cookie	Propiedad	–	Permite leer/modificar cookies.	Gestión básica de sesiones.
document.createTextNode()	Método	texto	Crea nodos de texto.	Inserción segura sin HTML.
document. queryCommandSupported()	Método	comando	Verifica comandos del editor.	Herramientas de edición.
document.activeElement	Propiedad	–	Elemento con foco.	Validación y accesibilidad.

Métodos adicionales de selección

Función / Propiedad	Tipo	Parámetros	Descripción breve	Uso típico
document.forms	Propiedad	–	Colección de formularios.	Acceso rápido a formularios.
document.images	Propiedad	–	Colección de imágenes.	Optimización y análisis.
document.links	Propiedad	–	Colección de `<a>` con href.	Gestión de enlaces.
document.scripts	Propiedad	–	Colección de `<script>`.	Control dinámico de scripts.
document.getElementsByName()	Método	nombre	Selección por atributo name.	Formularios y radios.
document.elementFromPoint(x, y)	Método	coordenadas	Devuelve el elemento bajo el puntero.	Detección avanzada de interacción.
document.getSelection()	Método	–	Devuelve el objeto de selección.	Herramientas de edición y marcado.
document.hasFocus()	Método	–	Comprueba si la ventana tiene foco.	Interacción y usabilidad.